

Deliverable 1: Requirements Table

Problem Statement #02 | Campus & Academic Operations - Automated Rubric Assignment Evaluator

A system that takes in batches of student code submissions, runs the configured test suite on each one in an isolated environment, and works out a rubric score from the results. It also hands out peer reviews on its own, which the Faculty Evaluator currently has to do by hand.

Primary Actors: Student, Faculty Evaluator. Supporting Actors: Peer Reviewer, Sandboxed Test Execution Engine, Notification Service.

Req ID	Type	Description	Priority	Acceptance Criteria (measurable pass/fail)	Rationale
FR-001	Functional	The system shall queue each uploaded student project archive, run the pre-configured unit test suite against it, and produce an itemised rubric score breakdown without the evaluator having to start the process.	High	Pass: a valid submission shows a per-criterion breakdown and its test case results within 10 seconds of upload, and the total shown matches the sum of the line items. Fail: the queue stops moving, or the total does not match the line items.	This covers the main job of the system. At the moment the evaluator has to run each project and add up the marks by hand.
FR-002	Functional	The system shall let a Faculty Evaluator write and activate a rubric before the submission window opens. Each rubric version records its criteria, the maximum marks for each one, and the test cases tied to it.	High	Pass: a rubric whose weights add up to 100% is saved with a new version number and used for every submission received after it goes live. Fail: weights that do not add up to 100% are accepted, or a live rubric is edited without a new version being made.	The rubric has to be settled before marking starts. If it can be edited halfway through, two students end up graded against different criteria.
FR-003	Functional	The system shall validate each uploaded archive for permitted file type, maximum size and required directory structure, and shall reject a non-conforming submission with a message naming the specific rule violated before it enters the evaluation queue.	High	Pass: a .rar file, a file over 25 MB, or an archive with no src/ folder is turned away within 3 seconds, and the message says which rule was broken. Fail: a bad archive gets onto the queue, or the message only says the file is invalid.	Bad uploads are the usual reason a shared queue gets stuck. Catching them at the door keeps the rest of the batch moving.
FR-004	Functional	The system shall automatically distribute each evaluated submission to a configured number of peer reviewers, guaranteeing that no student is assigned their own submission and that every submission receives an equal number of reviews.	Medium	Pass: for N submissions at k reviews each, the system produces N x k anonymous assignments in under 5 seconds, nobody is given their own work, and every submission gets exactly k reviewers. Fail: anyone is assigned their own submission, or the review counts come out uneven.	Handing out reviews manually is the overhead the system is supposed to remove. Splitting them evenly also stops anyone getting an unfair share.
FR-005	Functional	The system shall allow a Faculty Evaluator to override any automatically generated criterion score, shall require a written justification for each override, and shall retain both the original and revised scores in an immutable audit log.	Medium	Pass: an override with a reason of at least 20 characters changes the total, and the log stores the old score, the new score, who made the change and when. Fail: an override goes through with the reason left blank, or the old score is written over instead of kept.	Test cases do not always give credit for work that is partly right. If a mark gets changed there needs to be a record of who changed it, in case the student appeals.
NFR-001	Non-functional (Performance & Scalability)	The system shall sustain up to 100 concurrent project submissions without dropping any queue item and without the evaluation service exceeding a 1.5 GB resident memory footprint.	High	Pass: a load test of 100 submissions at once finishes with all 100 evaluated, nothing dropped from the queue, and peak memory at or under 1.5 GB. Fail: anything is dropped from the queue, or memory goes over 1.5 GB.	Almost everyone uploads in the last hour before the deadline, so the load is not spread out. Losing a submission at that point is the worst time for it to happen.
NFR-002	Non-functional (Security)	The system shall execute all student-supplied code inside an isolated sandbox with no outbound network access and a 60-second CPU limit per submission, and shall terminate any process that breaches either bound.	High	Pass: a submission that tries an outbound HTTP request, or one that loops forever, is stopped and marked 'Execution Halted (Policy Violation)', and nothing else on the queue is affected. Fail: the request gets out, or the runaway process slows down the rest of the queue.	The system runs code written by students, which cannot be trusted. Keeping it boxed in stops one bad submission from affecting everyone else on the same machine.

Peer Critique - Revisions Applied

Three things changed after the peer review. FR-001 first said the system should evaluate submissions quickly, which nobody could test, so it now has a 10 second limit and a check that the totals add up. FR-003 was changed so the rejection message names the rule that was broken. A message that only says the file is invalid gives the tester nothing to check. NFR-002 originally just said the system should be secure. It now names two limits a tester can actually try to break: no outbound network access, and a 60 second CPU ceiling.